# Measuring LoRA continued-pretraining throughput for a compute gate: probe design, five probe failures, and results on one A100

**PinoAI engineering note · ENG-001 · technical report**

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-09-30

## Abstract

Before choosing a base model for Filipino continued pretraining (CPT), the team fixed a compute gate in advance: a candidate is eligible only if its **measured** training throughput, with the method we would actually run, projects $T = 2 \times 10^9$ tokens within a GPU-hour budget B on one NVIDIA A100 80GB. This note describes the throughput probe used to measure that, the five ways the probe itself was wrong before it was right, and the results for four candidates: Gemma 4 E4B, Gemma 4 12B, Qwen3.5-9B-Base and Sailor2-8B.

The declared method is bf16 LoRA ($r = 64$, $\alpha = 128$, all linear layers) at sequence length 4,096 with non-reentrant gradient checkpointing, measured on synthetic tokens over 5 warm-up and 20 timed steps, at the fastest micro-batch in {1, 2, 4, 8} that fits in memory. Three implementation variants were measured on every model:

- **v1**, the Hugging Face loss;
- **v2**, a chunked cross-entropy that never materialises the full fp32 logits tensor;
- **v3**, v2 in a container image with the flash-linear-attention kernels for Qwen3.5's Gated DeltaNet layers.

Projected hours for 2B tokens at the registered method are 245.4 (Sailor2-8B), 278.2 (Qwen3.5-9B-Base, v3), 301.4 (Gemma 4 E4B) and 635.9 (Gemma 4 12B). **No candidate is within the budget.** Three engineering results carry beyond this decision:

1. **Two silent probe bugs inflated memory to near the 80 GB limit.** Caught out-of-memory (OOM) errors leaked memory between attempts, and the model was left in eval mode, where Hugging Face skips gradient checkpointing. The second one moved Gemma 4 E4B's peak at micro-batch 1 from 72.51 to 31.93 GiB.
2. **The chunked loss (v2) fixes memory, not speed.** It lowered peak memory by 5.2–33.9 GiB at equal micro-batch and let every model reach micro-batch 8 (12B: 4), but selected throughput moved only −0.2% to +4.2%.
3. **Kernel availability dominates for hybrid-attention models.** v3 raised Qwen3.5-9B-Base throughput by 83.7%, while the three non-DeltaNet models, run in the same image as controls, moved −1.7% to −3.5%. Repeating the Qwen v3 measurement gave a 4.4% different projection, which we take as the noise floor of every number here.

## 1. Setup

**Hardware.** One NVIDIA A100 80GB per model, on Modal (`gpu="A100-80GB"`). The provider can return either the SXM4 or the PCIe variant for that request. Every valid run below recorded `torch.cuda.get_device_name()` = `NVIDIA A100-SXM4-80GB` with 79.3 GiB total memory, and `_eng001.py` asserts this for every file it reads. One invalid early run (Section 3.1) landed on `NVIDIA A100 80GB PCIe`.

**Software.** torch 2.14.0, transformers 5.17.0, peft 0.21.0, accelerate 1.15.0, bitsandbytes 0.50.2, identical in every result file (asserted). Two container images:

- `image`: `debian_slim`, Python 3.12, the pinned packages above;
- `fast_image` (v3): `nvidia/cuda:13.0.0-devel-ubuntu22.04`, Python 3.12, the same pins plus flash-linear-attention 0.5.2, and an attempted build of causal-conv1d 1.7.0. The devel base is needed because the torch 2.14.0 wheels are built for CUDA 13.

**Models.** Pinned revisions and parameter counts, read from the result files:

Table 1: Candidates, pinned revisions, and parameter counts recorded by the probe after LoRA injection (total includes the LoRA parameters). Gemma 4 E4B's total includes its per-layer embeddings, so it is much larger than its effective compute size.

| model | HF id | revision | total params (incl. LoRA) | trainable (LoRA) | trainable share |
|---|---|---|---|---|---|
| Gemma 4 E4B | `google/gemma-4-E4B` | `411aa17b74` | 8.14B | 202.0M | 2.48% |
| Gemma 4 12B | `google/gemma-4-12B` | `023679ed35` | 12.22B | 263.7M | 2.16% |
| Qwen3.5-9B-Base | `Qwen/Qwen3.5-9B-Base` | `68c46c4b34` | 9.13B | 173.1M | 1.90% |
| Sailor2-8B | `sail/Sailor2-8B` | `fb9e3b997b` | 8.73B | 184.5M | 2.11% |

**Declared method.** Fixed in writing before any measurement:

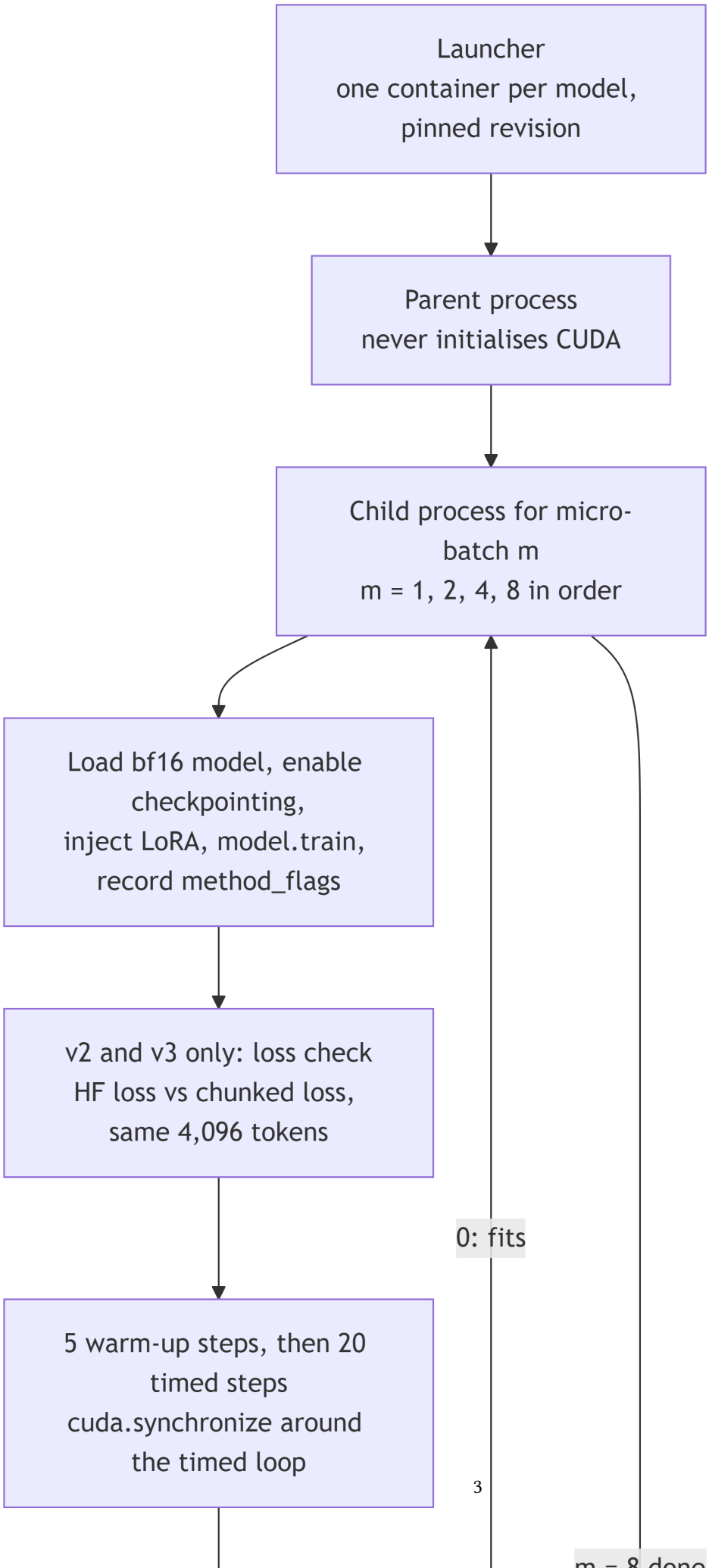
Table 2: The declared probe method. The table is the probe's argument defaults (scripts/training/gate_f_throughput.py), which every result file echoes under `settings`.

| setting | value |
|---|---|
| sequence length | 4,096 |
| adapter | LoRA r = 64, α = 128, dropout 0, `target_modules="all-linear"`, `task_type="CAUSAL_LM"` |
| weights / compute dtype | bf16 (`dtype="auto"` resolves to bf16 on A100; recorded as `dtype: bf16`) |
| quantisation | none (4-bit QLoRA was allowed only as a fallback and was never needed) |
| gradient checkpointing | on, `use_reentrant=False`; `use_cache=False`; `enable_input_require_grads()` |
| attention | `sdpa` (recorded per run) |
| optimiser | `torch.optim.AdamW`, lr 1e-4, LoRA parameters only |
| steps | 5 warm-up + 20 timed |
| data | uniform random token ids in [0, V − 1024), `torch.Generator` seed 0 |
| micro-batch | 1, 2, 4, 8, each in a fresh process, smallest first, stop at first OOM |
| targets | T = $2 \times 10^9$ tokens, GPU-hour budget B |

Synthetic data is deliberate: for a dense or hybrid decoder at fixed sequence length and without packing or padding, step time does not depend on token content. It also keeps any corpus off the rented machine. The top 1,024 ids are excluded to avoid special tokens.

## 2. Probe design

The probe (`gate_f_throughput.py`) is host-agnostic: it is a plain Python script that runs unchanged on any CUDA machine. The Modal launcher (`modal_gate_f.py`) only builds the image, starts one container per model and saves the returned JSON.


Launcher
one container per model,
pinned revision
Parent process
never initialises CUDA
Child process for micro-
batch m
m = 1, 2, 4, 8 in order
Load bf16 model, enable
checkpointing,
inject LoRA, model.train,
record method_flags
v2 and v3 only: loss check
HF loss vs chunked loss,
same 4,096 tokens
5 warm-up steps, then 20
timed steps
cuda.synchronize around
the timed loop
0: fits
m = 8 done

**Micro-batch search.** The parent process spawns one child per micro-batch in the order 1, 2, 4, 8, and stops at the first child that exits with an OOM. The parent never touches CUDA, so every child starts with an empty GPU. The number used for the gate is the **fastest** micro-batch that fit, which is not necessarily the largest (in practice it was always the largest; see Table 6 to Table 8).

**What is measured.** Inside the child, after warm-up, the 20 timed steps are bracketed by `torch.cuda.synchronize()` and `time.perf_counter()`. The script records:

- tokens/s = m × 4,096 × 20 / elapsed, and seconds/step = elapsed / 20. `_eng001.py` re-derives tokens/s from seconds/step for every attempt and asserts agreement.
- peak memory as `torch.cuda.max_memory_allocated()`, reset before warm-up (and again after the v2 loss check). This is allocated, not reserved, memory, so it understates what the caching allocator holds.
- the first and last training loss and a finite-loss flag. All attempts in all valid runs report finite losses.
- trainable and total parameter counts.
- `method_flags`: `model.training`, the base model's `is_gradient_checkpointing`, the resolved attention implementation, and, for DeltaNet models, the kernel that transformers actually bound (Section 3.5). These flags exist because two of the failures below were settings that were declared but silently not in effect.

**Projection.** hours = T / (tokens/s × 3,600), with $T = 2 \times 10^9$. There is no overhead factor, so the projection is a lower bound on wall time: it excludes data loading, evaluation, checkpoint I/O, restarts and the warm-up of a real run. `_eng001.py` recomputes it from tokens/s and asserts that it equals the probe's own `projection.hours_for_T`.

**Method v2: chunked cross-entropy.** The Hugging Face causal-LM loss materialises the logits for every position and upcasts them to fp32, a [m, 4,096, V] fp32 tensor. v2 calls the model with `logits_to_keep=1` and `output_hidden_states=True`, takes the final (already normed) hidden state, shifts it by one position like the HF loss, and computes the cross-entropy in chunks of 1,024 positions:

1. apply `lm_head` to the chunk and cast to fp32;
2. for Gemma, apply the model's `final_logit_softcapping` as tanh(x / cap) × cap;
3. compute the summed cross-entropy for the chunk.

Each chunk runs under `torch.utils.checkpoint` (non-reentrant), so its logits are recomputed in backward rather than stored. The chunk sums are added and divided by the number of targets. The math is the same as v1. Before timing, every child checks it: under `no_grad`, on the first 4,096-token row of the fixed batch, it computes both losses and records their absolute difference (Table 5).

**Method v3: fused DeltaNet kernels.** v3 is v2′s settings run in `fast_image`. Only Qwen3.5-9B-Base has Gated DeltaNet layers, so for the other three models v3 is a same-method, different-image **control**: any change there is an image effect, not a kernel effect.

# 3. Failures and fixes

Five problems were found in the probe before the numbers in Section 4 were accepted. Two produced invalid runs, which are kept, labelled, in the results folder.

Table 3: Gemma 4 E4B before and after the first two fixes, micro-batch 1. Runs 1 and 2 predate the `method_flags` field.

| run | procedure | GPU | attempts (mb:result) | mb 1 tok/s | mb 1 peak GiB | training_mode |
|---|---|---|---|---|---|---|
| run 1 | one process, order 8,4,2,1 | A100 80GB PCIe | 8:OOM, 4:OOM, 2:OOM, 1:ok | 1,921 | 72.48 | not recorded |
| run 2 | fresh process per mb, eval mode | A100-SXM4-80GB | 1:ok, 2:OOM | 1,971 | 72.51 | not recorded |
| v1 (fixed) | fresh process per mb, train mode | A100-SXM4-80GB | 1:ok, 2:ok, 4:OOM | 1,575 | 31.93 | True |

## 3.1 Caught OOMs leak memory into the next attempt

**Symptom.** The first run searched micro-batches 8, 4, 2, 1 inside one process, catching `OutOfMemoryError` and moving on. Micro-batches 8, 4 and 2 all failed on small allocations (320–640 MiB) with about 78 GiB already allocated by PyTorch. Micro-batch 1 fit, at a peak of 72.48 GiB.

**Root cause.** A caught `torch.cuda.OutOfMemoryError` keeps the failed attempt's tensors alive through the exception's traceback frames. So each later attempt started on a nearly full GPU, and a descending search can never find the true maximum.

**Fix.** One fresh process per micro-batch, ascending order, stop at the first OOM. The parent never initialises CUDA. An OOM in a child is written to its JSON and signalled by exit code 3.

**Verification.** Run 2 used the fixed search. It no longer showed cascading failures, but micro-batch 1 still peaked at 72.51 GiB and micro-batch 2 failed. That meant the leak was real but was not the main cause (3.2). Run 1 is also not comparable on speed, because it landed on the PCIe variant.

## 3.2 Gradient checkpointing was declared but never ran

**Symptom.** In run 2, with fresh processes, a 202M-parameter LoRA on Gemma 4 E4B still peaked at 72.51 GiB at micro-batch 1. That is far more than activations with checkpointing should need at 4,096 tokens.

**Root cause.** `from_pretrained` returns the model in eval mode. `gradient_checkpointing_enable()` sets the flag, but the Hugging Face layers only recompute activations when `self.gradient_checkpointing and self.training`, and nothing in the load path (checkpointing, input grads, `get_peft_model`) switched the model to train mode. So the backward pass kept every activation, and the declared method was not the one measured. Nothing warns about this; the loss still falls, and the step is even faster.

**Fix.** Call `model.train()` after `get_peft_model`. Record `method_flags` (`training_mode`, `gradient_checkpointing`, `attn_implementation`) in every result, so that a silently disabled setting is visible in the output rather than only in the memory numbers.

**Verification.** A same-model before/after comparison at micro-batch 1 (Table 3): peak memory fell from 72.51 to 31.93 GiB, and tokens/s fell from 1,971 to 1,575 (−20.1%, i.e. 1.25× the step time). That is the recompute cost of one extra forward pass, and it confirms that checkpointing is now active. Every valid result file records `training_mode: true` and `gradient_checkpointing: true`, and `_eng001.py` asserts both.

## 3.3 Full fp32 logits dominate memory at 262K vocabulary

**Symptom.** With checkpointing working, Gemma 4 E4B and 12B still hit OOM at micro-batch 4 under v1. In both cases the failed allocation was exactly 16.00 GiB.

**Root cause.** 16.00 GiB is exactly one fp32 tensor of shape [4, 4,096, 262,144]: 4 × 4,096 × 262,144 × 4 bytes = $2^{34}$ bytes: the fp32 logits of the HF loss for the whole micro-batch. The other OOMs fit the same pattern (Table 4). For each one we solved "requested bytes = rows × positions × V × 4" for V, and the result is consistent with a single fp32 logits tensor. The Qwen and Sailor2 values come out at about 248K and 152K. The v2 OOM of Gemma 4 12B at micro-batch 8 matches one fp32 chunk of the last 1,023 positions, which suggests that at that size even one chunk of the chunked loss no longer fits.

Table 4: Every OOM in the valid runs. "Implied V" solves requested bytes = rows × positions × V × 4 for V; the ± comes from the error message's two-decimal GiB rounding. This checks whether the OOM is consistent with one logits tensor; it does not read the model config.

| model | method | mb | requested GiB | if it is one logits tensor | implied V |
|---|---|---|---|---|---|
| Gemma 4 E4B | v1 | 4 | 16.00 | [4, 4096, V] fp32 | 262,144 ± 82 |
| Gemma 4 12B | v1 | 4 | 16.00 | [4, 4096, V] fp32 | 262,144 ± 82 |
| Gemma 4 12B | v2 | 8 | 7.99 | [8, 1023, V] fp32 | 262,072 ± 164 |
| Gemma 4 12B | v3 | 8 | 7.99 | [8, 1023, V] fp32 | 262,072 ± 164 |
| Qwen3.5-9B-Base | v1 | 8 | 30.31 | [8, 4096, V] fp32 | 248,300 ± 41 |
| Sailor2-8B | v1 | 8 | 18.55 | [8, 4096, V] fp32 | 151,962 ± 41 |

**Fix.** Method v2 (Section 2), which never holds more than one [m, 1,024, V] fp32 chunk.

**Verification.** Before timing, every v2/v3 child compared v1 and v2 losses on the same 4,096 tokens (Table 5). The largest relative difference is $3.4 \times 10^{-5}$ (Gemma 4 E4B, $6.4 \times 10^{-4}$ absolute on a loss of 18.48). We attribute it to bf16 `lm_head` matmuls on differently shaped inputs; we did not investigate further. For a throughput probe, agreement to about $10^{-5}$ relative is enough to show that v2 computes the same objective. After v2, peak memory at equal micro-batch fell by 5.2–33.9 GiB (Table 9), and every model fit micro-batch 8 except Gemma 4 12B (4).

Table 5: Loss-equality check, run under `no_grad` in every child before timing, on the first 4,096-token row of the fixed seed-0 batch. It is identical in every child of a run. The Qwen3.5 v1 loss itself differs between runs (see text).

| model | run | v1 loss (HF) | v2 loss (chunked) | abs diff | rel diff |
|---|---|---|---|---|---|
| Gemma 4 E4B | v2 | 18.482643 | 18.482006 | 6.4e-04 | 3.4e-05 |
| Gemma 4 E4B | v3 | 18.482643 | 18.482006 | 6.4e-04 | 3.4e-05 |
| Gemma 4 12B | v2 | 14.179514 | 14.179721 | 2.1e-04 | 1.5e-05 |
| Gemma 4 12B | v3 | 14.179514 | 14.179721 | 2.1e-04 | 1.5e-05 |
| Qwen3.5-9B-Base | v2 | 13.816250 | 13.816250 | 0.0e+00 | 0.0e+00 |
| Qwen3.5-9B-Base | v3 | 13.816804 | 13.816805 | 9.5e-07 | 6.9e-08 |
| Qwen3.5-9B-Base | v3 re-check | 13.816319 | 13.816321 | 1.9e-06 | 1.4e-07 |
| Sailor2-8B | v2 | 13.118582 | 13.118584 | 1.9e-06 | 1.5e-07 |
| Sailor2-8B | v3 | 13.118582 | 13.118584 | 1.9e-06 | 1.5e-07 |

One side observation: the Qwen3.5 v1 loss on the same tokens was 13.816250 in the v2 image and 13.816804 / 13.816319 in the two v3 runs. Changing the image changes the DeltaNet code path (3.4), and two runs of the same v3 image still differ at the $10^{-5}$ relative level. That is consistent with non-deterministic reductions in the fused kernel. It does not matter for throughput, but it does matter for anyone expecting bitwise-reproducible losses.

## 3.4 The DeltaNet fast path was missing

**Symptom.** Under v2, Qwen3.5-9B-Base reached 1,087 tokens/s. Sailor2-8B, a model of similar size (Table 1) under the same method, reached 2,263. At micro-batch 1 under v1 the gap was larger still (785 versus 2,144), and Qwen's v2 throughput climbed the most steeply with micro-batch (+25.9% from 1 to 8, Table 10). That points to a slow, poorly batched component rather than to the matmuls.

**Root cause.** Qwen3.5 interleaves Gated DeltaNet (linear-attention) layers with full attention. When the optional flash-linear-attention and causal-conv1d packages are absent, transformers silently falls back to a pure-PyTorch reference implementation of the chunked gated delta rule and of the short convolution.

**Fix.** Method v3: the same settings in `fast_image`, with flash-linear-attention 0.5.2 (Triton). The causal-conv1d 1.7.0 CUDA extension **failed to build** because compiler detection failed, not because of the kernel itself: `torch.utils.cpp_extension._check_cuda_version` read the host clang++ version as 0.0.0 and rejected it as below the minimum (7.0) for CUDA 13.0 (`gate-f-modal-v3.log`, about line 738; line 748 is only the build-failure marker). The build step is guarded (`|| echo CAUSAL_CONV1D_BUILD_FAILED`), so the image built anyway, with only the FLA chunk rule available.

**Verification.** Three things support the fix:

1. Qwen3.5 throughput rose by 83.7% (1,087 → 1,997 tokens/s).
2. The three control models moved −1.7% to −3.5% in the same image.
3. The re-check in 3.5 showed which kernel was actually bound.

## 3.5 The kernel-engagement check looked in the wrong place

**Symptom.** The registered v3 run for Qwen3.5 recorded `method_flags.deltanet.chunk_rule: null` and `causal_conv1d: null`, even though the throughput had nearly doubled.

**Root cause.** transformers 5.17 does not bind the kernels on the layer instance. At module import, `use_kernel_func_from_hub_with_fallback` wraps the module-level functions `torch_chunk_gated_delta_rule` and `causal_conv1d_fn`, and keeps the resolved implementation in the wrapper's closure (`implementation`). The first check read instance attributes, found nothing, and reported null.

**Fix.** For the first module whose class name contains `DeltaNet`, the check now looks up its defining module in `sys.modules`, reads each wrapped function's `__closure__` by `co_freevars` name, and records the resolved implementation's `__module__.__name__`.

**Verification.** A re-check run with the fixed flag recorded `chunk_rule: fla.ops.gated_delta_rule.chunk.chunk_gated_delta_rule` and `causal_conv1d: transformers.models.qwen3_5.modeling_qwen3_5.causal_conv1d_fn`. In other words, the FLA chunk rule en-

gaged, and the short convolution used the torch fallback, as the failed build predicts. The re-check is kept as a sensitivity row. The registered v3 number stays the earlier run, which is the slower and so the conservative one.

## 4. Results

All rows below were measured on `NVIDIA A100-SXM4-80GB` with `training_mode`, `gradient_checkpointing` = true and `sdpa` attention. Cells show tokens/s with peak allocated GiB in parentheses. "OOM" marks the first micro-batch that did not fit (the search stops there), and "-" marks a micro-batch that was not tried.

Table 6: Method v1 (Hugging Face loss).

| model | method | mb 1 | mb 2 | mb 4 | mb 8 | selected mb | selected tok/s | h for 2B |
|---|---|---|---|---|---|---|---|---|
| Gemma 4 E4B | v1 | 1,575 (31.9) | 1,769 (47.1) | OOM | - | 2 | 1,769 | 314.1 |
| Gemma 4 12B | v1 | 801 (40.8) | 863 (56.3) | OOM | - | 2 | 863 | 643.9 |
| Qwen3.5-9B-Base | v1 | 785 (31.1) | 1,018 (43.5) | 1,065 (68.4) | OOM | 4 | 1,065 | 521.7 |
| Sailor2-8B | v1 | 2,144 (25.9) | 2,230 (33.8) | 2,268 (49.5) | OOM | 4 | 2,268 | 245.0 |

Table 7: Method v2 (chunked fp32 cross-entropy, same loss).

| model | method | mb 1 | mb 2 | mb 4 | mb 8 | selected mb | selected tok/s | h for 2B |
|---|---|---|---|---|---|---|---|---|
| Gemma 4 E4B | v2 | 1,519 (22.0) | 1,689 (27.2) | 1,794 (37.6) | 1,843 (58.4) | 8 | 1,843 | 301.4 |
| Gemma 4 12B | v2 | 802 (30.9) | 851 (36.4) | 874 (47.6) | OOM | 4 | 874 | 635.9 |
| Qwen3.5-9B-Base | v2 | 864 (22.6) | 1,005 (26.6) | 1,053 (34.5) | 1,087 (50.4) | 8 | 1,087 | 511.1 |
| Sailor2-8B | v2 | 2,160 (20.7) | 2,219 (23.5) | 2,257 (28.9) | 2,263 (39.8) | 8 | 2,263 | 245.4 |

Table 8: Method v3 (v2 in the fast-kernel image). For Gemma and Sailor2 the kernels are unused, so these rows are image controls. The Qwen re-check row is the sensitivity measurement from 3.5.

| model | method | mb 1 | mb 2 | mb 4 | mb 8 | selected mb | selected tok/s | h for 2B |
|---|---|---|---|---|---|---|---|---|
| Gemma 4 E4B | v3 | 1,530 (22.0) | 1,674 (27.2) | 1,766 (37.6) | 1,810 (58.4) | 8 | 1,810 | 306.9 |
| Gemma 4 12B | v3 | 793 (30.9) | 837 (36.4) | 859 (47.6) | OOM | 4 | 859 | 646.8 |
| Qwen3.5-9B-Base | v3 | 1,972 (22.6) | 1,963 (26.6) | 1,992 (34.5) | 1,997 (50.4) | 8 | 1,997 | 278.2 |
| Qwen3.5-9B-Base | v3 re-check | 2,017 (22.6) | 2,033 (26.6) | 2,071 (34.5) | 2,088 (50.4) | 8 | 2,088 | 266.0 |
| Sailor2-8B | v3 | 2,083 (20.7) | 2,142 (23.5) | 2,176 (28.9) | 2,184 (39.8) | 8 | 2,184 | 254.4 |

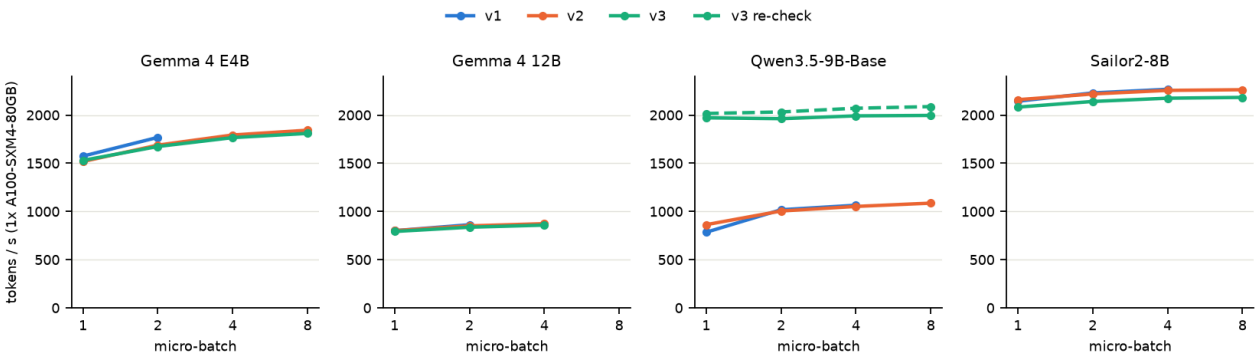



Figure 1: Tokens/s per micro-batch, by method. Only Qwen3.5-9B-Base changes materially between v2 and v3; the dashed line is the v3 re-check.

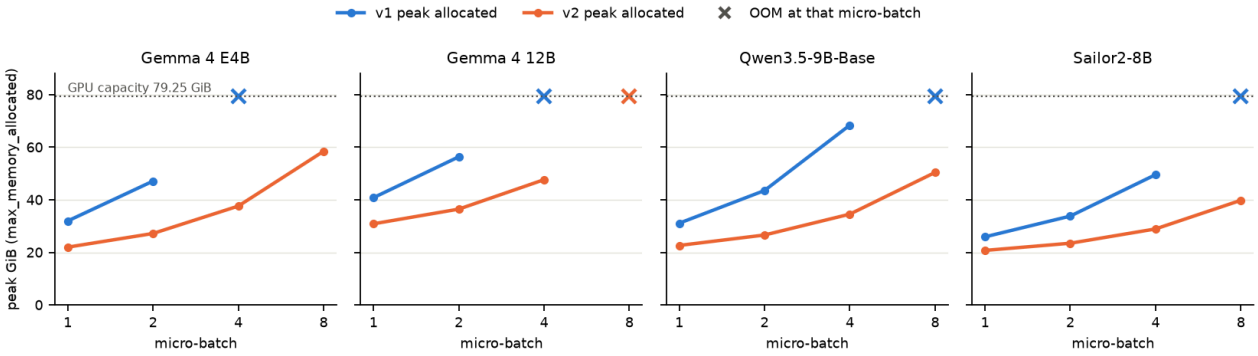



Figure 2: Peak allocated memory per micro-batch under v1 and v2; × marks the micro-batch that ran out of memory, plotted at the 79.25 GiB capacity reported in the error.

## 4.1 v2 fixes memory, not speed

Table 9: Method deltas at the selected micro-batch. The peak saving compares v1 and v2 at the same micro-batch, over the micro-batches where v1 fit.

| model | v1 -> v2 peak saved at same mb (GiB) | v1 -> v2 tok/s (selected) | v2 -> v3 tok/s (selected) | role in v2 -> v3 |
|---|---|---|---|---|
| Gemma 4 E4B | 10.0-19.9 | +4.2% | −1.8% | control |
| Gemma 4 12B | 9.9-19.9 | +1.3% | −1.7% | control |
| Qwen3.5-9B-Base | 8.5-33.9 | +2.1% | +83.7% | kernel target |
| Sailor2-8B | 5.2-20.6 | −0.2% | −3.5% | control |

Table 10: Throughput gain from larger micro-batches once memory allows.

| model | method | largest mb that fits | tok/s gain mb 1 -> largest | tok/s gain mb 2 -> largest |
|---|---|---|---|---|
| Gemma 4 E4B | v2 | 8 | +21.4% | +9.2% |
| Gemma 4 E4B | v3 | 8 | +18.3% | +8.1% |
| Gemma 4 12B | v2 | 4 | +9.0% | +2.7% |
| Gemma 4 12B | v3 | 4 | +8.3% | +2.6% |
| Qwen3.5-9B-Base | v2 | 8 | +25.9% | +8.1% |
| Qwen3.5-9B-Base | v3 | 8 | +1.2% | +1.7% |
| Sailor2-8B | v2 | 8 | +4.8% | +2.0% |
| Sailor2-8B | v3 | 8 | +4.8% | +2.0% |

v2 removed the logits bottleneck and let micro-batches grow, but throughput barely moved (−0.2% to +4.2% at the selected micro-batch). Beyond micro-batch 2, every model gains at most 9.2%. Step time scales almost linearly with micro-batch (for example, Sailor2-8B v2 takes 1.90 s per step at micro-batch 1 and 14.48 s at 8), so these runs are compute-bound once the full logits tensor is gone. Larger micro-batches will not rescue the gate on this GPU. The memory v2 frees (about 21–40 GiB of the 79.25 GiB unused at the selected micro-batch, Table 7) could instead pay for turning

checkpointing off. The team deliberately did not do that for the gate verdict: it would be a method change made after seeing the results. It remains a legitimate speed-up for the real run, once a base is chosen, but it must be re-measured.

## 4.2 Controls isolate the kernel effect

The three non-DeltaNet models ran the identical v2 method in both images. Their v2 → v3 change (−1.7%, −1.8%, −3.5%) is therefore an image effect: CUDA 13 devel base versus `debian_slim`, the same pinned wheels, and a different container. Qwen3.5′s +83.7% corresponds to a net kernel effect of **+86.9% to +90.4%** after adjusting by the control range. v3 did not change peak memory for any model: the peaks are identical to v2 for the three controls and within 0.01 GiB for Qwen3.5. With the kernel, Qwen3.5′s throughput is also flat in micro-batch (+1.2% from 1 to 8), which suggests that the reference DeltaNet path was what had made it batch-sensitive.

## 4.3 Run-to-run spread

Table 11: Qwen3.5-9B-Base v3: registered run versus re-check, same method, same image, same GPU variant, a different container.

| mb | v3 tok/s | v3 re-check tok/s | difference | peak GiB (v3 / re-check) |
|---|---|---|---|---|
| 1 | 1,972 | 2,017 | +2.3% | 22.62 / 22.62 |
| 2 | 1,963 | 2,033 | +3.6% | 26.58 / 26.58 |
| 4 | 1,992 | 2,071 | +4.0% | 34.52 / 34.52 |
| 8 | 1,997 | 2,088 | +4.6% | 50.39 / 50.39 |
| selected | | | | 278.2 h vs 266.0 h (-4.4%) |

The re-check was faster at every micro-batch (+2.3% to +4.6%), and the projection moved from 278.2 to 266.0 hours (−4.4%). The only intended difference was the engagement flag, which does not run in the timed loop. We therefore treat about ±4% as the run-to-run noise floor of a single probe run, and apply it to every number in this note. This is also why Sailor2-8B's v1 (245.0 h) and v2 (245.4 h) projections cannot be told apart. The registered figure per model is the faster of its v2 and v3 runs: Sailor2-8B 245.4, Qwen3.5-9B-Base 278.2, Gemma 4 E4B 301.4 and Gemma 4 12B 635.9 hours. With a ±4% band, no candidate comes near the budget: the closest, at 245.4 hours, would need 22.7% more throughput.

## 4.4 Utilisation (estimate)

Table 12: **Estimate, not a measurement.** Model FLOP/s = 6N × tokens/s, where 6N counts 2N forward, 2N backward (activation gradients only, since the base weights are frozen and have no weight-gradient matmuls), and 2N recompute under checkpointing. N is the total parameter count including embeddings, which overstates FLOPs (strongly so for Gemma 4 E4B's per-layer embeddings); attention-score FLOPs at 4,096 tokens are ignored, which understates them.

| model | best of v2/v3 | tok/s | N (total) | ~TFLOP/s (6N x tok/s) | ~MFU vs 312 TFLOP/s |
|---|---|---|---|---|---|
| Gemma 4 E4B | v2 | 1,843 | 8.14B | 90 | 29% |
| Gemma 4 12B | v2 | 874 | 12.22B | 64 | 21% |
| Qwen3.5-9B-Base | v3 | 1,997 | 9.13B | 109 | 35% |
| Sailor2-8B | v2 | 2,263 | 8.73B | 119 | 38% |

Under this approximation the runs use roughly 20–40% of the A100′s dense bf16 peak. The Gemma 4 E4B figure is an upper bound, because its per-layer embeddings inflate N. There is headroom (fused or chunked kernels for the loss, selective rather than full checkpointing, or no checkpointing now that memory allows), but none of it was allowed to change the gate verdict.

## 5. Cost

Table 13: GPU time recorded in the result files, (5 + 20) × seconds/step summed over every attempt that fit, priced at the provider's A100 80GB list rate. This is a lower bound: model download and load, image builds, OOM attempts and container start-up are not in the files.

| runs | timed GPU minutes (warmup + timed steps) | at $0.000694/s |
|---|---|---|
| pre-fix runs 1-2 (E4B) | 1.8 | $0.07 |
| method v1 (4 models) | 26.4 | $1.10 |
| method v2 (4 models) | 64.1 | $2.67 |
| method v3 (4 models) | 53.4 | $2.22 |
| v3 re-check (Qwen) | 12.4 | $0.51 |
| **total timed** | **158.0** | **$6.58** |

The recorded warm-up and timed steps add up to 158.0 GPU-minutes, or $6.58 at list price. The provider's billing summary for the month to date (read on 2026-09-30, about 09:20 UTC) showed **$13.25** of ephemeral compute. That covered all of this note's runs plus unrelated evaluation jobs and two orphaned launches, and the whole amount was absorbed by the free monthly credit (billed cost $0.00). The gap between $6.58 and $13.25 is model loading, image builds (the causal-conv1d compile alone runs for minutes), failed attempts and the other jobs. The probe design choice with the largest cost impact is the fresh process per micro-batch, which re-loads the model up to four times per run.

## 6. Reproducibility

Table 14: Files.

| item | path |
|---|---|
| probe (host-agnostic) | `scripts/training/gate_f_throughput.py` |
| launcher, images `image` and `fast_image` | `scripts/training/modal_gate_f.py` |
| v1 / v2 / v3 results | `experiments/results/gate-f/modal/<slug>.json`, `<slug>.v2.json`, `<slug>.v3.json` |
| v3 re-check | `experiments/results/gate-f/modal/qwen-qwen3-5-9b-base.v3check.json` |
| invalid pre-fix runs | `google-gemma-4-e4b.run1-leaky-oom-search.json`, `google-gemma-4-e4b.run2-eval-mode.json` (same folder) |
| container logs | `experiments/results/gate-f-modal-*.log` |
| tables and figures of this note | `experiments/reports/_eng001.py` |

Commands (the launcher's `modal` client is not in the project environment, so it is brought in with `uv`):

```
# v1, all four models (one container per model, results to experiments/results/gate-f/modal/)
uv run --with modal==1.6.0 modal run scripts/training/modal_gate_f.py
# v2
uv run --with modal==1.6.0 modal run scripts/training/modal_gate_f.py --extra "--method v2"
--tag v2
# v3 = v2 settings in the fast-kernel image
uv run --with modal==1.6.0 modal run scripts/training/modal_gate_f.py --extra "--method v2"
--tag v3 --fast
# any other CUDA host, one model
python scripts/training/gate_f_throughput.py --model google/gemma-4-E4B --revision
411aa17b74 --method v2 --out gate_f.json
# regenerate this note's tables and figures (asserts method, GPU string, projections)
python experiments/reports/_eng001.py
```

The v3 JSONs record `settings.method: "v2"` (the probe has no v3 flag; v3 is an image) and a top-level `method_label: "v3"`. Model revisions are pinned by full commit hash in the launcher.

## 7. Limitations

- **One GPU type, one run per cell.** Every number is a single 20-step measurement on the A100-SXM4-80GB. The only repeat (4.3) differed by up to 4.6%, and the other cells carry similar uncertainty. PCIe A100s, H100s or multi-GPU setups were not measured and should not be extrapolated from these numbers.
- **Lower-bound projection.** Hours assume the timed-step rate holds for 2B tokens. Data loading, evaluation, checkpointing, restarts and learning-rate warm-up are excluded.
- **Synthetic tokens, no packing.** Real CPT with document packing and attention masks, or variable-length batches, can run at a different speed. This matters most for the DeltaNet layers, whose kernels handle sequence boundaries differently.
- **Short timed window.** 5 warm-up steps may not reach steady state for Triton autotuning in the v3 image, and 20 steps do not expose thermal or clock throttling.
- **Peak memory is `max_memory_allocated`.** It excludes allocator fragmentation and reserved-but-unused memory. Two OOMs (12B v2/v3 at micro-batch 8) failed with 10.74 GiB reserved but unallocated, so the true fitting boundary may be improvable with `expandable_segments`. This was not tested.
- **Partial v3.** causal-conv1d did not build, so v3 is FLA's chunk rule plus the torch convolution. A fully fused v3 might be faster for Qwen3.5.
- **The declared method was held fixed on purpose.** Checkpointing off, other LoRA targets, sequence packing or other kernels could change the ranking. They were excluded from the verdict so that the gate stayed pre-registered, not because they are unimportant.
- **Utilisation is an estimate** (Table 12), resting on the stated FLOP approximation and the vendor peak.

## Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

**AI assistance.** Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.

## References